Week 6: Product Review and Rating Management System

Organic Grocery Store Management System

The Product Review and Rating Management System is used to collect customer feedback about products. Customers can give ratings from 1 to 5 stars and write reviews about the products they purchase. This system helps store customer opinions, calculate average product ratings, identify highly rated products, and find products that need improvement. It helps the company understand customer satisfaction and improve product quality.

1. Create Review and Rating Tables

```
SQL> connect C##MOGS
Enter password:
Connected.
SQL> CREATE TABLE Review (Review_ID NUMBER PRIMARY KEY,Customer_ID NUMBER,Pr
oduct_ID NUMBER,Rating NUMBER CHECK (Rating BETWEEN 1 AND 5),Review_Comment
VARCHAR2(200), Review_Date DATE,FOREIGN KEY (Customer_ID) REFERENCES Custome
r(Customer_ID),FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID));

Table created.
```

2.Insert values

```
Table created.

SQL> INSERT INTO Review VALUES (1, 1, 101, 5, 'Excellent quality', DATE '202
6-10-20');

1 row created.

SQL> INSERT INTO Review VALUES (2, 2, 102, 4, 'Very good product', DATE '202
6-10-21');

1 row created.

SQL> INSERT INTO Review VALUES (3, 3, 103, 3, 'Average quality', DATE '2026-
10-22');

1 row created.

SQL> INSERT INTO Review VALUES (4, 1, 104, 5, 'Very fresh', DATE '2026-10-23
');

1 row created.

SQL> INSERT INTO Review VALUES (5, 2, 101, 2, 'Needs improvement', DATE '202
6-10-24');

1 row created.
```

```
SQL> SELECT * FROM REVIEW;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
         1           1        101          5
Excellent quality
20-OCT-26

         2           2        102          4
Very good product
21-OCT-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------

         3           3        103          3
Average quality
22-OCT-26

         4           1        104          5
Very fresh

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
23-OCT-26

         5           2        101          2
Needs improvement
```

3. Update and Delete Reviews

Update a review comment:

```
SQL> UPDATE Review SET Review_Comment = 'Excellent product' WHERE Review_ID
= 1;

1 row updated.
```

Delete an inappropriate review:

```
SQL> DELETE FROM Review WHERE Review_ID = 5;

1 row deleted.
```

4. Retrieve Product Review Details

Find 1: Display all reviews for a product

```
SQL> SELECT * FROM Review WHERE Product_ID = 101;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
----
REVIEW_DA
---------
         1           1        101          5
Excellent product
20-OCT-26
```

Find 2: Display customer names with reviews

```
SQL> SELECT c.First_Name, r.Product_ID, r.Rating, r.Review_Comment FROM Cust
omer c JOIN Review r ON c.Customer_ID = r.Customer_ID;

FIRST_NAME                                         PRODUCT_ID     RATING
-------------------------------------------------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------
----
Arun                                                      101          5
Excellent quality

Priya                                                     102          4
Very good product

Kavin                                                     103          3
Average quality


FIRST_NAME                                         PRODUCT_ID     RATING
-------------------------------------------------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------------------------
----
Arun                                                      104          5
Very fresh

Priya                                                     101          2
Needs improvement
```

Find 3: Products having the maximum number of reviews

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews FROM Review GROUP BY Produ
ct_ID ORDER BY Total_Reviews DESC;

PRODUCT_ID TOTAL_REVIEWS
---------- -------------
       101             2
       103             1
       104             1
       102             1
```

Find 4: Display recent customer feedback

```
SQL> SELECT * FROM Review ORDER BY Review_Date DESC;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------
----
REVIEW_DA
---------
         5           2        101          2
Needs improvement
24-OCT-26

         4           1        104          5
Very fresh
23-OCT-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------
----
REVIEW_DA
---------

         3           3        103          3
Average quality
22-OCT-26

         2           2        102          4
Very good product

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------------
----
REVIEW_DA
---------
21-OCT-26
```

```
REVIEW_DA
---------
         5           2         101           2
Needs improvement
24-OCT-26

         4           1         104           5
Very fresh
23-OCT-26

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
----
REVIEW_DA
---------

         3           3         103           3
Average quality
22-OCT-26

         2           2         102           4
Very good product

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
----
REVIEW_DA
---------
21-OCT-26

         1           1         101           5
Excellent quality
20-OCT-26
```

Find 5: Reviews with ratings above 4

```
SQL> SELECT * FROM Review WHERE Rating > 4;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID     RATING
---------- ----------- ---------- ----------
REVIEW_COMMENT
--------------------------------------------------
REVIEW_DA
---------
         1           1        101          5
Excellent quality
20-OCT-26

         4           1        104          5
Very fresh
23-OCT-26
```

5. Calculate Average Product Ratings

Find 1: Average rating for each product

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY P
roduct_ID;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       102              4
       101            3.5
       104              5
       103              3
```

Find 2: Number of reviews per product

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews FROM Review GROUP BY Produ
ct_ID;

PRODUCT_ID TOTAL_REVIEWS
---------- -------------
       102             1
       101             2
       104             1
       103             1
```

Find 3: Highest-rated products

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY P
roduct_ID ORDER BY Average_Rating DESC;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       104              5
       102              4
       101            3.5
       103              3
```

Find 4: Products with average ratings above 4

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY P
roduct_ID HAVING AVG(Rating) > 4;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       104              5
```

6. Product Rating Reports

Report 1: Product Rating Analysis

Display product name, number of reviews, and average rating.

```
SQL> SELECT p.Product_Name, COUNT(r.Review_ID) AS Total_Reviews, NVL(AVG(r.R
ating), 0) AS Average_Rating FROM Product p LEFT JOIN Review r ON p.Product_
ID = r.Product_ID GROUP BY p.Product_ID, p.Product_Name;

PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Organic Apple
            2            3.5

Brown Rice
            1              5

Organic Banana
            1              4


PRODUCT_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVIEWS AVERAGE_RATING
------------- --------------
Green Tea
            0              0

Organic Carrot
            1              3
```

Report 2: Customer Feedback Analysis

Find 1: Most reviewed products

```
SQL> SELECT Product_ID, COUNT(*) AS Total_Reviews FROM Review GROUP BY Produ
ct_ID ORDER BY Total_Reviews DESC;

PRODUCT_ID TOTAL_REVIEWS
---------- -------------
       101             2
       103             1
       104             1
       102             1
```

Find 2: Highly rated products

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY P
roduct_ID HAVING AVG(Rating) >= 4;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       102              4
       104              5
```

Find 3: Products requiring improvement

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Review GROUP BY P
roduct_ID HAVING AVG(Rating) < 3;

no rows selected
```

Report 3: Rating Distribution

Find 1: Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings FROM Review WHERE Rating = 5;

FIVE_STAR_RATINGS
-----------------
                2
```

Find 2: Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS Four_Star_Ratings FROM Review WHERE Rating = 4;

FOUR_STAR_RATINGS
-----------------
                1
```

Find 3: Number of low-rated reviews (1 or 2 stars)

```
SQL> SELECT COUNT(*) AS Low_Rated_Reviews FROM Review WHERE Rating <= 2;

LOW_RATED_REVIEWS
-----------------
                1
```

Find 4: Complete rating distribution

```
SQL> SELECT Rating, COUNT(*) AS Total FROM Review GROUP BY Rating ORDER BY R
ating DESC;

    RATING      TOTAL
---------- ----------
         5          2
         4          1
         3          1
         2          1
```